📘 Multidimensional Arrays

A **multi-dimensional array** can be termed as an array of arrays that stores homogeneous data in tabular form.

📘 Representation

✅ 2-D Array (Matrix Form)

- Represented using **rows and columns**

Accessed using:
a[row][column]

-

Example Structure:

Row 0 → a[0][0] a[0][1] a[0][2]
Row 1 → a[1][0] a[1][1] a[1][2]
Row 2 → a[2][0] a[2][1] a[2][2]
Row 3 → a[3][0] a[3][1] a[3][2]
Row 4 → a[4][0] a[4][1] a[4][2]

✅ 3-D Array

- Extension of 2D array

Represented as:
a[i][j][k]

-
- Used for storing data in **3 dimensions (cube format)**

📘 Initializing Multidimensional Arrays

Methods of Initialization:

- Providing values during declaration
- Using nested loops

📘 Example: 2D Array in C

```C/C++
#include <stdio.h>
int main() {
    // Declare and initialize a 2D array
    int matrix[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };
    // Access and print elements
    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 4; ++j) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

📘 Example: 3D Array in C

```c
#include <stdio.h>
int main() {
    // Declare and initialize a 3D array
    int cube[2][3][4] = {
        {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}},
        {{13, 14, 15, 16}, {17, 18, 19, 20}, {21, 22, 23, 24}}
    };
    // Access and print elements
    for (int i = 0; i < 2; ++i) {
        for (int j = 0; j < 3; ++j) {
            for (int k = 0; k < 4; ++k) {
                printf("%d ", cube[i][j][k]);
            }
            printf("\n");
        }
        printf("\n");
    }
}
```

📘 Input and Display 2D Array Using Loops

```c
#include <stdio.h>
int main() {
    int rows, columns;
    printf("Enter number of rows and columns: ");
    scanf("%d %d", &rows, &columns);
    int matrix[rows][columns];
    // Input values
```

```
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < columns; ++j) {
            printf("Enter value for element at position [%d][%d]:
", i, j);
            scanf("%d", &matrix[i][j]);
        }
    }
    // Display array
    printf("\nEntered 2D Array:\n");
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < columns; ++j) {
            printf("%d\t", matrix[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

📘 Accessing Elements in Multidimensional Arrays

✅ Two-Dimensional Indexing

Elements are accessed using:
x[row][column]

-

Example:
x[0][0], x[0][1], x[0][2]
x[1][0], x[1][1], x[1][2]
x[2][0], x[2][1], x[2][2]

-

✅ Three-Dimensional Indexing

Elements are accessed using:
x[i][j][k]

-
- Represents:
    - i → layer
    - j → row
    - k → column

📘 Memory Layout of Multidimensional Arrays

✅ Row-Major Order

- Elements are stored **row by row in memory**
- First row is stored completely, then second row, and so on

Example:

For:

1 2 3 4
5 6 7 8
9 10 11 12

Stored in memory as:

1 2 3 4 5 6 7 8 9 10 11 12

📘 C Program: Assign Values & Print Memory Addresses

```C/C++
#include <stdio.h>
int main() {
    int rows = 3;
    int columns = 4;
    int matrix[rows][columns];
    // Assign values to the 2D array
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < columns; ++j) {
            matrix[i][j] = i * columns + j + 1;
        }
    }
    // Print addresses of elements
    printf("Addresses of elements in the 2D array:\n");
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < columns; ++j) {
            printf("matrix[%d][%d]: %p\n", i, j,
(void*)&matrix[i][j]);
        }
    }
    return 0;
}
```

📘 Applications of 2D Arrays

- Matrix Operations
- Image Processing
- Student Grades Storage

📘 Example: Student Grades Program

```C/C++
#include <stdio.h>
int main() {
    // Student grades example
    int grades[3][5] = {
        {85, 90, 92, 88, 89},
        {78, 85, 80, 87, 82},
        {92, 88, 90, 85, 95}
    };
    // Display student grades
    printf("Student Grades:\n");
    for (int i = 0; i < 3; ++i) {
        printf("Student %d: ", i + 1);
        for (int j = 0; j < 5; ++j) {
            printf("%d ", grades[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

📘 Passing Multidimensional Arrays to Functions

- Passing a multidimensional array to a function in C involves specifying the **array dimensions** in the function parameter list.

✅ Example in C

```C/C++
#include <stdio.h>
// Function prototype with array parameter
void function_name(int rows, int columns, int
array[rows][columns]) {
    // Function code here
}
int main() {
    int matrix[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };
    // Call the function
    function_name(3, 4, matrix);
    return 0;
}
```

📘 Applications of 3D Arrays

- Weather Modeling
- Geological Modeling
- Medical Treatment Planning
- Array of Cubes or Cuboid Representation
- Robotics and Path Planning

📘 Dynamic Memory Allocation for Multidimensional Arrays

✅ Concept

- Dynamic Memory Allocation (DMA) is used to allocate memory at runtime.
- Functions used:
  - `malloc()`
  - `calloc()`
  - `realloc()`
- For multidimensional arrays:
  👉 Memory must be allocated **for each dimension separately**

✅ Example: Dynamic Allocation of 2D Array

```C/C++
#include <stdio.h>
#include <stdlib.h>
int main() {
    int rows = 3;
    int columns = 4;
    // Allocate memory for rows
    int** matrix = (int**)malloc(rows * sizeof(int*));
    // Check allocation
    if (matrix == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }
    // Allocate memory for columns
    for (int i = 0; i < rows; ++i) {
        matrix[i] = (int*)malloc(columns * sizeof(int));
        if (matrix[i] == NULL) {
            printf("Memory allocation failed.\n");
            return 1;
        }
    }
    // Initialize and print values
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < columns; ++j) {
            matrix[i][j] = i * columns + j + 1;
```

```
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

printf("%d ", matrix[i][j]);

📘 Common Pitfalls

- ❌ Contiguous Memory Assumption
- ❌ Incorrect Array Indexing
- ❌ Row-Column Confusion
- ❌ Size Mismatch
- ❌ Incomplete Initialization
- ❌ Assuming Equal Row Sizes

📘 Best Practices

- ✓ Understand Array Dimensions
- ✓ Use Descriptive Variable Names
- ✓ Use `sizeof` for Memory Calculations
- ✓ Handle Dynamic Memory Carefully
- ✓ Use Functions for Modularity